Implementing and Evaluating Monte Carlo Tree Search for Connect Four

Author: Andrew Falcone
Course: ME 569
Date: June 5, 2026

Abstract

This project implements and evaluates Monte Carlo Tree Search (MCTS) for Connect Four under practical compute constraints. Connect Four is deterministic and adversarial, but its branching factor and board size make exhaustive search much less convenient than in Tic Tac Toe. The project therefore uses Upper Confidence bounds applied to Trees (UCT) to choose actions by repeatedly selecting promising tree paths, expanding new actions, simulating games with rollout policies, and backpropagating terminal or estimated values to earlier nodes. The experiments compare baseline agents and MCTS variants using fixed simulation budgets, exploration constants, and rollout policies. Three rollout strategies are considered: random rollouts, heuristic-guided rollouts that use immediate tactical checks and a hand-built board evaluator, and value-function cutoffs based on an offline-trained NumPy value model. The current results suggest that stronger compute budgets generally improve MCTS, heuristic-guided rollouts provide the most reliable improvement against the baseline agents, and the preliminary value model is promising but not yet tuned enough to replace stronger rollout policies. Overall, the project shows both the strengths of MCTS for deterministic games and the practical importance of rollout quality, search budget, and runtime cost.

1. Introduction

Search and reinforcement learning have a long history in game-playing systems because games provide clear states, actions, outcomes, and performance metrics. Temporal-difference learning showed that a value function could be learned from self-play and used directly for action selection, as in TD-Gammon, which reached near-expert backgammon performance without relying on a database of expert moves [1]. Monte Carlo planning approaches later addressed large search spaces by estimating action quality through simulated futures rather than exhaustive tree traversal [2]. Upper Confidence bounds applied to Trees (UCT) uses a bandit-style selection rule to balance exploration of under-sampled actions with exploitation of actions that have already produced strong outcomes [2].

More recent systems combine search with learned representations. AlphaGo Zero used self-play, Monte Carlo Tree Search, and a neural network that produced both move probabilities and value estimates, removing the need for supervised expert-game pretraining and traditional random rollouts [3]. AlphaZero extended the same general learning-and-search framework to chess, shogi, and Go, demonstrating that MCTS guided by learned policy and value estimates can generalize across deterministic adversarial games [4]. These systems motivate the direction of this project, but reproducing their full training scale is outside the scope of a course project. Instead, this project studies the same basic planning tradeoff in a smaller Connect Four setting: how much benefit comes from more search, better rollout policies, and simple value estimates under fixed compute budgets.

Connect Four is a useful test environment for this question because it is simple to describe but still large enough to make naive full-width search expensive. The board has only seven legal action choices at most, yet the number of possible move sequences grows quickly as pieces are placed. Unlike stochastic games such as backgammon, Connect Four is deterministic: the same action from the same board always leads to the same next state. It is also adversarial: each player is trying to choose actions that reduce the other player's chance of winning. These properties make the game a good fit for studying UCT search, rollout policies, and compute-budget tradeoffs [2].

This project uses Tic Tac Toe as a smaller debugging testbed before scaling to Connect Four. Tic Tac Toe shares the same deterministic, two-player structure, but the state space is small enough to inspect directly and compare against exact minimax values. That makes it useful for verifying the MCTS interface, tree-search data flow, and visualization tools. Connect Four is the more meaningful target because the larger search space forces MCTS to make approximate decisions from incomplete search rather than simply enumerating all relevant outcomes.

The goal of the project is to build a reusable Connect Four software stack and use it to evaluate MCTS behavior. The implementation includes a core game-state object, baseline agents, a game-independent UCT MCTS implementation, Gymnasium-style environments, an interactive pygame visualizer, and scripted experiments for comparing agent strength and runtime cost. The main experimental questions are how MCTS performance changes with simulation budget, how sensitive UCT is to the exploration constant, and whether random, heuristic-guided, or value-function-assisted rollouts provide the best tradeoff between playing strength and compute cost.

2. Background

2.1 Connect Four Environment

The Connect Four environment uses a 6 row by 7 column board. Empty cells are stored as `0`, Red pieces are stored as `+1`, and Yellow pieces are stored as `-1`. The core state keeps the board in this fixed Red/Yellow representation and separately tracks the current player. A move is represented by choosing one of the seven columns. If the column is legal, the piece falls to the lowest empty row in that column. A column becomes illegal when its top row is already occupied.

A game terminates when one player connects four pieces horizontally, vertically, or diagonally. If the board fills without a winner, the game is a tie. The core state exposes legal actions, an action mask, the last move, the current player, terminal status, and the winner. This allows the same game rules to be used by the interactive GUI, agents, automated experiments, and Gymnasium-style wrappers.

The reward and value convention is perspective based. Calling `result_for(player)` returns `+1` if that player wins, `-1` if that player loses, and `0` for a tie or non-terminal position. MCTS stores values from the root player's perspective, so each rollout result is interpreted relative to the player who started the current search. The Gymnasium wrappers can also return observations from a player perspective by multiplying the fixed Red/Yellow board by the selected player value. This makes the player-to-move or learner's own pieces appear as `+1` in the observation, but the underlying core board is not negated when turns change.

2.2 Monte Carlo Tree Search

Monte Carlo Tree Search builds a partial search tree from the current game state. Each node stores a game state, the action that reached it, child nodes, a visit count, and a cumulative value. Instead of exhaustively expanding the entire game tree, MCTS spends its compute budget on repeated simulations. Each simulation has four phases:

1. **Selection:** Starting from the root, repeatedly choose a child node using a rule that balances exploitation and exploration.
2. **Expansion:** When a node still has untried legal actions, add one of those actions as a new child.
3. **Rollout:** Simulate from the new or selected state until a terminal state is reached, a rollout depth limit is reached, or a value-function cutoff is used.
4. **Backpropagation:** Add the rollout value to every node on the selected path and increment their visit counts.

This project uses the UCT selection rule introduced by Kocsis and Szepesvari [2]. For a candidate child node, the score is:

```
score = mean_value + c * sqrt(log(parent_visits) / child_visits)
```

The `mean_value` term is exploitation: it favors actions that have produced good outcomes so far. The square-root term is exploration: it favors actions with fewer visits. The constant `c` controls the balance between those two terms. A larger `c` makes search explore less-visited actions more aggressively, while a smaller `c` makes search rely more heavily on the current value estimates.

The implementation stores values from the root player's perspective. At nodes where it is the root player's turn, selection favors children with high root-player value. At opponent nodes, selection favors children with low root-player value because the opponent is assumed to choose actions that hurt the root player. After the search budget is spent, the selected move is the root child with the highest visit count, using mean value and action order only as tie breakers.

The default rollout policy samples random legal actions. Additional experiments replace random rollouts with heuristic-guided rollouts or a value-function cutoff. A time limit can also be used with an iteration cap; in that case, search stops when either the time cap or iteration cap is reached first.

2.3 Baseline Agents

Several baseline agents are used to make the experiments easier to interpret:

- `random`: uniformly samples legal columns.
- `bad`: uses the same board-scoring function as the heuristic agent but deliberately chooses among the lowest-scoring actions.
- `heuristic`: first takes an immediate winning move if one exists, then blocks the opponent's immediate winning move if needed, and otherwise chooses among the highest-scoring one-ply actions.

- `mcts`: runs UCT MCTS with configurable iteration count, exploration constant, rollout policy, optional time limit, and optional value-function cutoff.
- `first-legal`: chooses the leftmost legal column and is mainly useful as a deterministic debugging baseline.

The heuristic board evaluator rewards center-column control because center pieces participate in more potential four-in-a-row lines. It also rewards open two-in-a-row and three-in-a-row windows for the current player, with a much larger score for immediate or near-immediate wins. Opponent threats are penalized with similar logic, and opponent three-in-a-row windows are penalized more strongly than the player's own three-in-a-row reward because failing to block an immediate threat usually loses the game.

These agents serve two purposes. First, they provide sanity checks: stronger agents should generally beat weaker agents. Second, they provide fixed opponents for measuring how MCTS changes as simulation budget, exploration constant, and rollout policy change.

3. Methods

3.1 Software Architecture

The project is organized as an installable Python package under `src/connect4`, with command-line entry points defined in `pyproject.toml`. The same core game and agent interfaces are reused by the GUI, scripted experiments, and Gymnasium-style environments. This kept the implementation small enough to inspect while still allowing the experiments to be run repeatedly from the command line.

Main files:

- `src/connect4/core.py`: game state and rules.
- `src/connect4/mcts.py`: game-independent MCTS.
- `src/connect4/agents.py`: baseline and MCTS agents.
- `src/connect4/env.py`: Gymnasium-style environment.
- `src/connect4/pygame_app.py`: interactive visualizer.
- `src/connect4/scripts/evaluate_agents.py`: automated experiments.
- `src/connect4/value_model.py`: lightweight NumPy value model.
- `src/connect4/scripts/generate_value_dataset.py`: value-data generation.
- `src/connect4/scripts/train_value_model.py`: value-model training.

3.2 Tic Tac Toe Testbed

Tic Tac Toe was used as a smaller testbed before Connect Four. It uses the same two-player MCTS interface, but the game is small enough that exact minimax values can be computed and displayed in the GUI. This made it easier to verify that selection, expansion, rollout, and backpropagation behaved sensibly before applying the same MCTS code to a larger Connect Four search space.

The main Tic Tac Toe checks were:

```
tictactoe-mcts --iterations 300 --seed 0 --verbose
tictactoe-play
```

3.3 Experimental Protocol

All reported experiments used fixed random seeds so the runs could be reproduced. Baseline matchups used seed `0` and 100 games for each ordered Red/Yellow pairing. The main MCTS-vs-heuristic sweeps used seed `0` and 100 games per side, for 200 total games per condition. In those sweep experiments, MCTS played both Red and Yellow against the fixed heuristic opponent to reduce the effect of first-player advantage. The preliminary value-model cutoff runs were smaller because of runtime constraints: they used 30 games per side, for 60 total games per condition.

Metrics:

- win rate
- tie rate
- average moves per game
- seconds per game
- seconds per move

- MCTS seconds per move
- simulations per MCTS move

Confidence interval:

```
p_hat = wins / n
SE = sqrt(p_hat * (1 - p_hat) / n)
95% CI = p_hat +/- 1.96 * SE
```

Note: This report uses the normal approximation for win-rate confidence intervals.

3.4 Experiments

Experiment 1: Baseline Agent Matchups Purpose:

This experiment checks whether the implemented agents have a sensible strength ordering before using them in more focused MCTS experiments. The expected ordering is that the deliberately bad heuristic should lose to most agents, random should beat the bad agent but lose to stronger agents, the heuristic agent should beat simple baselines, and higher-budget MCTS should generally beat lower-budget MCTS.

Command:

```
connect4-evaluate matchups \
  --agents random,bad,heuristic,mcts:100,mcts:500 \
  --games 100 \
  --seed 0 \
  --progress-every 10
```

Expected output:

- Compact table of win rates.
- Use this as a sanity-check table, not the main result figure.

Experiment 2: Simulation Budget Sweep Purpose:

This experiment measures how MCTS performance changes as simulations per move increase while the exploration constant, rollout policy, opponent, and color-swapping protocol stay fixed.

Command:

```
connect4-evaluate c-sweep \
  --iterations 10,50,100,500,1000 \
  --exploration-weights sqrt2 \
  --games-per-side 100 \
  --seed 0 \
  --csv outputs/connect4_budget_sweep.csv \
  --plot outputs/connect4_budget_sweep.png \
  --progress-every 10
```

Controlled variables:

- Exploration constant fixed at `sqrt(2)`.
- Rollout policy fixed to random.
- Opponent fixed to heuristic.
- MCTS plays both first and second player.

Output files:

- `outputs/connect4_budget_sweep.csv`
- `outputs/connect4_budget_sweep.png`

Interpretation:

The sweep is intended to show both playing strength and runtime cost. If MCTS is working correctly, increasing the simulation budget should generally improve win rate, but the gain per additional simulation should eventually shrink while seconds per move continues to increase.

Experiment 3: Exploration Constant Sweep Purpose:

This experiment measures sensitivity to the UCT exploration constant `c`. The goal is not to prove a single best constant for all settings, but to see whether too little or too much exploration hurts performance at practical simulation budgets.

Command:

```
connect4-evaluate c-sweep \
  --iterations 100,500 \
  --exploration-weights 0.25,0.5,1.0,sqrt2,2.0,4.0 \
  --games-per-side 100 \
  --seed 0 \
  --progress-every 10
```

Controlled variables:

- Rollout policy fixed to random.
- Opponent fixed to heuristic.
- Iteration budgets tested at 100 and 500.
- MCTS plays both first and second player.

Output files:

- `outputs/connect4_c_sweep.csv`
- `outputs/connect4_c_sweep.png`

Interpretation:

The comparison is interpreted separately at 100 and 500 iterations because the best exploration setting can depend on how much search is available. Very low `c` can over-commit to early rollout noise, while very high `c` can spend too many visits on weak actions.

Experiment 4: Rollout Policy Comparison Purpose:

This experiment compares the default random rollout policy with a heuristic-guided rollout policy. A smaller preliminary run also tests whether an offline value model can replace part of the rollout with a cheaper learned value estimate.

Command without value model:

```
connect4-evaluate rollout-policy-sweep \
  --policies random,heuristic \
  --iterations 100,500 \
  --games-per-side 100 \
  --seed 0 \
  --progress-every 10
```

Optional value-model workflow:

```
connect4-generate-value-data \
  --games 200 \
  --seed 0 \
  --output outputs/value_data/connect4_value_dataset_200g.npz \
  --metadata outputs/value_data/connect4_value_dataset_200g_metadata.json
connect4-train-value-model \
  --dataset outputs/value_data/connect4_value_dataset_200g.npz \
  --output outputs/value_models/connect4_value_model_200g.npz \
  --epochs 100 \
  --seed 0
connect4-evaluate rollout-policy-sweep \
  --policies random,heuristic,value \
  --value-model outputs/value_models/connect4_value_model_200g.npz \
  --value-depth 8 \
```

```
--iterations 100 \
--games-per-side 30 \
--seed 0 \
--csv outputs/connect4_rollout_policy_sweep_with_value_small.csv \
--plot outputs/connect4_rollout_policy_sweep_with_value_small.png \
--progress-every 10
```

Output files:

- `outputs/connect4_rollout_policy_sweep.csv`
- `outputs/connect4_rollout_policy_sweep.png`
- `outputs/connect4_rollout_policy_sweep_with_value_small.csv`
- `outputs/connect4_rollout_policy_sweep_value_depth0_small.csv`

Interpretation:

The main comparison is performance per simulation and per second. Heuristic rollouts are expected to improve rollout quality but cost more time per rollout. The value-function cutoff is expected to be faster than full rollouts, but its strength depends on the quality of the learned value estimate.

4. Results

4.1 Baseline Matchups

Table 1 summarizes the baseline matchup results from `outputs/connect4_agent_matchups.csv`.

Red agent	Yellow agent	Red win	Yellow win	Tie	Avg moves
random	bad	100.0%	0.0%	0.0%	19.94
random	heuristic	0.0%	100.0%	0.0%	10.36
random	mcts:100	2.0%	98.0%	0.0%	13.26
random	mcts:500	0.0%	100.0%	0.0%	12.12
bad	random	0.0%	100.0%	0.0%	21.58
bad	heuristic	0.0%	100.0%	0.0%	8.00
bad	mcts:100	0.0%	100.0%	0.0%	11.82
bad	mcts:500	0.0%	100.0%	0.0%	9.58
heuristic	random	100.0%	0.0%	0.0%	7.44
heuristic	bad	100.0%	0.0%	0.0%	7.00
heuristic	mcts:100	75.0%	24.0%	1.0%	16.17
heuristic	mcts:500	14.0%	86.0%	0.0%	25.34
mcts:100	random	100.0%	0.0%	0.0%	10.64
mcts:100	bad	100.0%	0.0%	0.0%	9.66
mcts:100	heuristic	55.0%	45.0%	0.0%	18.71
mcts:100	mcts:500	18.0%	81.0%	1.0%	26.30
mcts:500	random	100.0%	0.0%	0.0%	9.80
mcts:500	bad	100.0%	0.0%	0.0%	8.12
mcts:500	heuristic	92.0%	7.0%	1.0%	16.04
mcts:500	mcts:100	88.0%	8.0%	4.0%	21.04

Interpretation:

The baseline ordering is mostly consistent with expectations. The deliberately bad heuristic loses to every other agent in the table, while the random agent beats the bad agent but loses badly to the heuristic and MCTS agents. The heuristic agent beats random and bad, but it is weaker than the 500-iteration MCTS agent in both color orders. The 100-iteration MCTS agent is competitive with the heuristic agent, but the result depends on color order: it wins 55% as Red against Yellow heuristic, while heuristic as Red beats Yellow `mcts:100` 75% of the time. The stronger MCTS agent also beats the weaker MCTS agent in both direct ordered pairings, winning 88% as Red and 81% as Yellow.

4.2 Simulation Budget

Table 2 summarizes the simulation-budget sweep from `outputs/connect4_budget_sweep.csv`. The corresponding plot is `outputs/connect4_budget_sweep.png`. Confidence interval values are shown in percentage points.

Iterations	Games	Win % (+/- CI)	Tie %	Avg moves	MCTS sec/move
10	200	0.5 (+/- 1.0)	0.0	10.20	0.005
50	200	17.5 (+/- 5.3)	0.0	13.14	0.023
100	200	40.0 (+/- 6.8)	0.5	18.68	0.043
500	200	87.5 (+/- 4.6)	0.0	20.00	0.189
1000	200	95.0 (+/- 3.0)	1.0	19.15	0.349

Key observations:

- Very small budgets perform poorly against the heuristic baseline: 10 iterations wins only 0.5%, and 50 iterations wins 17.5%.
- Performance improves sharply once the search has enough simulations. The win rate rises to 40.0% at 100 iterations, 87.5% at 500 iterations, and 95.0% at 1000 iterations.
- Runtime increases with the simulation budget. The improvement from 500 to 1000 iterations is smaller than the improvement from 100 to 500 iterations, suggesting diminishing returns in this opponent setting.

4.3 Exploration Constant

Table 3 summarizes the exploration-constant sweep from `outputs/connect4_c_sweep.csv`. The corresponding plot is `outputs/connect4_c_sweep.png`. Confidence interval values are shown in percentage points.

Iterations	c	Games	Win % (+/- CI)	Tie %	MCTS sec/move
100	0.25	200	21.0 (+/- 5.6)	0.5	0.041
100	0.5	200	33.0 (+/- 6.5)	0.5	0.041
100	1	200	43.5 (+/- 6.9)	1.0	0.042
100	sqrt(2)	200	32.5 (+/- 6.5)	0.5	0.042
100	2	200	39.0 (+/- 6.8)	0.0	0.044
100	4	200	26.5 (+/- 6.1)	0.5	0.043
500	0.25	200	53.0 (+/- 6.9)	0.5	0.153
500	0.5	200	75.5 (+/- 6.0)	0.5	0.171
500	1	200	84.5 (+/- 5.0)	0.5	0.184
500	sqrt(2)	200	88.5 (+/- 4.4)	0.5	0.181
500	2	200	83.0 (+/- 5.2)	0.0	0.180
500	4	200	71.5 (+/- 6.3)	1.0	0.185

Key observations:

- At 100 iterations, `c = 1.0` gives the highest measured win rate at 43.5%, although the confidence intervals overlap several nearby settings.
- At 500 iterations, `sqrt(2)` gives the highest measured win rate at 88.5%, with `c = 1.0` and `c = 2.0` also performing strongly.
- Very low and very high exploration values perform worse at 500 iterations. This suggests that some exploration is important, but excessive exploration wastes simulations on less promising moves.

4.4 Rollout Policy

Table 4 summarizes the main rollout-policy comparison from `outputs/connect4_rollout_policy_sweep.csv`. The corresponding plot is `outputs/connect4_rollout_policy_sweep.png`. Confidence interval values are shown in percentage points.

Iterations	Rollout	Games	Win % (+/- CI)	Tie %	Avg moves	MCTS sec/move
100	random	200	43.0 (+/- 6.9)	0.5	17.06	0.043
100	heuristic	200	94.0 (+/- 3.3)	0.0	19.51	0.661
500	random	200	86.0 (+/- 4.8)	1.0	19.36	0.191
500	heuristic	200	98.0 (+/- 1.9)	0.0	21.84	2.748

Table 5 summarizes the preliminary value-model cutoff runs from the two small value-cutoff CSV files in `outputs/`. Confidence interval values are shown in percentage points.

Rollout	Games	Win % (+/- CI)	Tie %	MCTS sec/move
random	60	33.3 (+/- 11.9)	0.0	0.040
heuristic	60	91.7 (+/- 7.0)	0.0	0.637
value depth 8	60	46.7 (+/- 12.6)	1.7	0.024
value depth 0	60	53.3 (+/- 12.6)	0.0	0.004

Key observations:

- Heuristic-guided rollouts greatly improve playing strength. At 100 iterations, the win rate increases from 43.0% with random rollouts to 94.0% with heuristic rollouts.
- The strength improvement has a large runtime cost. At 500 iterations, heuristic rollouts reach 98.0% win rate, but they require 2.748 MCTS seconds per move compared with 0.191 seconds for random rollouts.
- The preliminary value cutoff is much faster than heuristic rollouts, but it is not yet as strong. In the small 100-iteration runs, the value cutoff reaches 46.7% at depth 8 and 53.3% at depth 0, compared with 91.7% for heuristic rollouts.

4.5 Runtime Cost

Table 6 summarizes runtime and win-rate tradeoffs across the budget, rollout-policy, and preliminary value-cutoff experiments.

Condition	Simulations/move	MCTS sec/move	MCTS win rate
random rollout, 100 iters	100	0.043	40.0%
random rollout, 500 iters	500	0.189	87.5%
random rollout, 1000 iters	1000	0.349	95.0%
heuristic rollout, 100 iters	100	0.661	94.0%
heuristic rollout, 500 iters	500	2.748	98.0%
value cutoff depth 8, 100 iters	100	0.024	46.7%
value cutoff depth 0, 100 iters	100	0.004	53.3%

Interpretation:

The runtime results show that there is no single best configuration without considering compute budget. Random rollouts are cheap enough that increasing the simulation count from 100 to 1000 remains practical and improves win rate from 40.0% to 95.0%. Heuristic rollouts produce stronger play per simulation, reaching 94.0% at only 100 iterations, but each simulation is much more expensive. The value cutoff runs are the fastest, especially with depth 0, but the current model is not strong enough to match heuristic-guided rollouts.

5. Discussion

The clearest result is that MCTS benefits strongly from additional simulations when the rollout policy is fixed. Against the heuristic baseline, random-rollout MCTS is very weak at 10 and 50 iterations, becomes competitive around 100 iterations, and becomes strong by 500 to 1000 iterations. This matches the expected behavior of UCT: early estimates are noisy, but additional simulations make the visit distribution more informative.

The exploration-constant results are more nuanced. At 100 iterations, `c = 1.0` performed best in this sample, while at 500 iterations `sqrt(2)` performed best. The differences between nearby constants should not be over-interpreted because several confidence intervals overlap. However, the sweep does show that extremely low and high values are worse than middle values in this setting. This supports using `sqrt(2)` as a reasonable default, but not as a universally optimal value.

The rollout-policy comparison shows the largest practical tradeoff. Heuristic-guided rollouts make MCTS much stronger per simulation because they avoid many obviously poor random continuations and include immediate tactical checks. However, they are much more expensive per move. This means the best rollout policy depends on whether the system is constrained by simulation count or wall-clock time. A slow but informed rollout can beat a cheap random rollout at the same iteration count, but a cheaper rollout may allow many more iterations in the same time budget.

The value-network cutoff is promising but preliminary. It is much faster than heuristic rollouts and can cheaply estimate non-terminal positions, but the current value model was trained on only 200 generated games and was evaluated with a smaller sample size. The value cutoff did not yet match heuristic rollouts, so the current model should be treated as a proof of concept rather than a final replacement for rollout simulation.

First-player advantage and opponent choice are important limitations. Some direct matchups change noticeably depending on whether the agent plays Red or Yellow, so color-swapped evaluations are more reliable than one-sided matchups. Also, most sweeps use win rate against the heuristic agent as the main metric. That is useful for a controlled comparison, but it does not prove optimal play or general strength against every possible opponent.

Limitations:

- Connect Four is not solved by this implementation.
- MCTS searches are stochastic.
- Experiments use finite samples.
- Neural value function, if included, is trained from generated self-play outcomes, not perfect-play labels.
- Runtime depends on hardware.

6. Summary And Future Work

This project implemented a reusable Connect Four environment, baseline agents, a game-independent UCT MCTS implementation, GUI visualizers, experiment scripts, and a preliminary value-function workflow. Tic Tac Toe was used as a smaller debugging environment before scaling the same MCTS interface to Connect Four.

The main finding is that MCTS performance depends strongly on search budget and rollout quality. With random rollouts, more simulations consistently improved performance against the heuristic baseline, but runtime increased with the budget. Heuristic-guided rollouts produced much stronger play at the same iteration count, but at a much higher seconds-per-move cost. The value-function cutoff was fast and potentially useful, but the current model was too preliminary to replace heuristic-guided rollouts.

Future work should focus on reusing computation and improving learned guidance. Reusing MCTS trees between moves and adding transposition tables would reduce wasted search. Training stronger value or policy networks on larger datasets could make the value-cutoff approach more competitive. More experiments against additional opponents and tactical benchmark positions would also make the strength estimates more reliable.

Future work ideas:

- Reuse MCTS trees between moves.
- Add transposition tables.
- Train stronger value/policy networks.
- Compare against tabular Q-learning on reduced board sizes.
- Evaluate tactical benchmark positions.
- Increase experiment repetitions for tighter confidence intervals.

Code And Data Availability

Code repository:

https://github.com/falconeaj1/ME_569_RL

Generated outputs:

- `outputs/connect4_agent_matchups.csv`
- `outputs/connect4_budget_sweep.csv`
- `outputs/connect4_budget_sweep.png`
- `outputs/connect4_c_sweep.csv`
- `outputs/connect4_c_sweep.png`
- `outputs/connect4_rollout_policy_sweep.csv`
- `outputs/connect4_rollout_policy_sweep.png`
- `outputs/connect4_rollout_policy_sweep_with_value_small.csv`
- `outputs/connect4_rollout_policy_sweep_with_value_small.png`
- `outputs/connect4_rollout_policy_sweep_value_depth0_small.csv`
- `outputs/connect4_rollout_policy_sweep_value_depth0_small.png`
- `outputs/value_data/connect4_value_dataset_200g.npz`
- `outputs/value_data/connect4_value_dataset_200g_metadata.json`
- `outputs/value_models/connect4_value_model_200g.npz`
- `outputs/value_models/connect4_value_model_200g.json`

Contributions

Andrew Falcone:

- Project scope and implementation.
- Connect Four environment and agents.
- MCTS implementation.
- Evaluation scripts.
- GUI visualizers.
- Experiments and report writing.
- Review of generated code, outputs, tables, and final report text.

Codex/ChatGPT assistance:

- Used for wording and organization of the report.
- Used for guidance on experiment design and interpretation.
- Used to run project commands, format tables, and help inspect generated outputs.
- All code, experiment results, and report text were reviewed by Andrew Falcone.

Bibliography

[1] G. Tesauro, "Temporal Difference Learning and TD-Gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58-68, 1995.

[2] L. Kocsis and C. Szepesvari, "Bandit Based Monte-Carlo Planning," in *European Conference on Machine Learning (ECML)*, 2006.

[3] D. Silver et al., "Mastering the Game of Go without Human Knowledge," *Nature*, vol. 550, pp. 354-359, 2017.

[4] D. Silver et al., "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," arXiv:1712.01815, 2018.